

Chapitre II :

Eléments de base de PL/SQL



► SQL: Le langage des bases de données relationnelles

► Séparer les données et les traitements pour manipuler:

- Des **bases de données complexes** (un grand nombre de tables) et **volumineuses**;
- Dans des **environnements hétérogènes**: matériels et logiciels

► Il est normalisé (normes SQL2 et SQL3) **mais** ...;

► C'est un **langage algébrique** basé sur le calcul relationnel;

► C'est un **langage déclaratif**: on décrit le résultat (pas l'algorithme);

► Il est composé de trois parties:

- Le LDD : Langage de définition des données (Create, Alter, Drop),
- Le LMD: Langage de manipulation des données,
- Le LCD: Langage de contrôle des données.



- ▶ **PL/SQL** (Procedural Language extension to SQL) est une **extension procédurale à SQL**.
- ▶ PL/SQL est un langage complet. Il comporte:
 - La partie LID (Langage d'Interrogation des Données) de SQL(Select),
 - La partie LMD (Langage de Manipulation des données) de SQL(Update, Insert, Delete),
 - La gestion des transactions (Commit, Rollback, Savepoint),
 - Les fonctions standards de SQL (to_char, to_date, upper, etc.),
 - **Plus une partie procédurale** (des variables, des boucles: IF,While, des curseurs, traitement des erreurs...).



► SQL vs PL/SQL

emp_id	first_name	last_name	salary
1	John	Doe	5000
2	Jane	Smith	6500
3	Alice	Johnson	5500
4	Kali	Geo	4500

► Pour augmenter le salaire de 10% pour les employés ayant le salaire <5000

SQL :

```
UPDATE employees SET salary = salary * 1.1 WHERE salary < 5000;
```

► Pour augmenter le salaire selon les conditions spécifiées:

- Si le salaire <5000, augmenter de 10%
- Si le 5000<=salaire<6000, augmenter de 7%
- Si le salaire >=6000, augmenter de 5%

PL SQL :

```
PROCEDURE :  
BEGIN  
  IF ...THEN  
    ELSIF ...THEN  
  ELSE  
  END IF;  
END;
```



► Le langage PL/SQL ne comporte pas de commandes du LDD telles que : **Create, Alter, Drop, Rename**

► Il ne comporte pas aussi des commandes du LCD telles que : **Grant et Revoke**

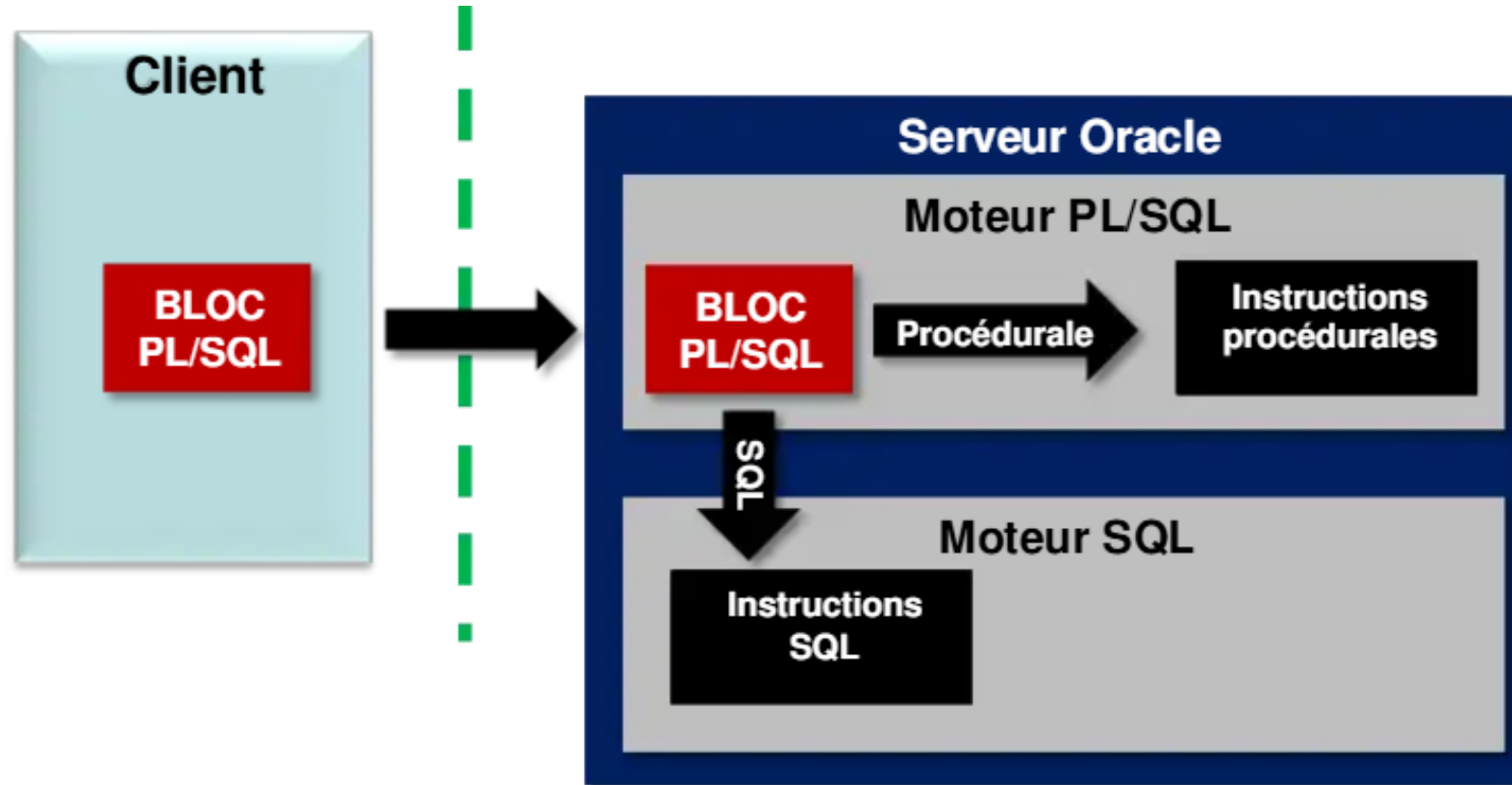
➔ **Seuls les commandes SQL: Select, Insert, Update et Delete qu'on peut les trouver dans un bloc PL/SQL**

► PL/SQL peut être utilisé dans de nombreux produits oracle parmi lesquels :SQL*PLUS, isqlplus, SQL*Forms, Oracle Reports, etc.

PL/SQL est donc un langage algorithmique standard : c'est un mariage entre l'assertionnel et le procédural



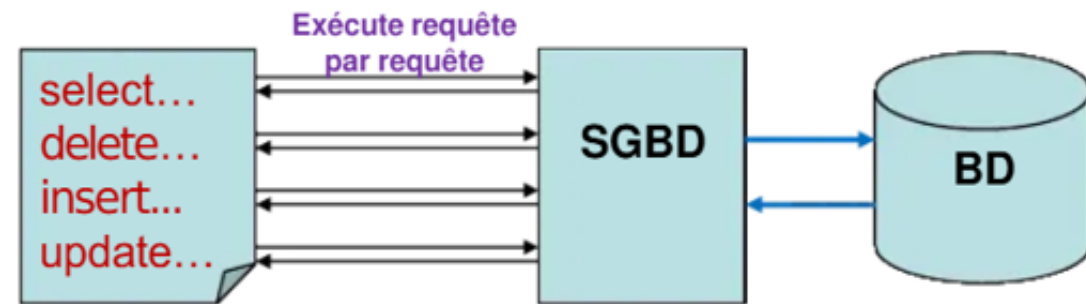
- ▶ Le moteur de BD Oracle, coordonne tous les appels de la base.
 - ▶ **SQL** et **PL/SQL** comportent chacun un « moteur d'exécution » associé, respectivement **SQL STATEMENT EXECUTOR** et **PROCEDURAL STATEMENT EXECUTOR**
 - ▶ Lorsque le serveur reçoit un appel pour exécuter un programme PL/SQL, la version compilée du programme est chargée en mémoire puis exécutée par les moteurs PL/SQL et SQL.
- Le moteur PL/SQL gère les structures mémoire et le flux logique du programme, tandis que le moteur SQL transmet à la base les requêtes de données.



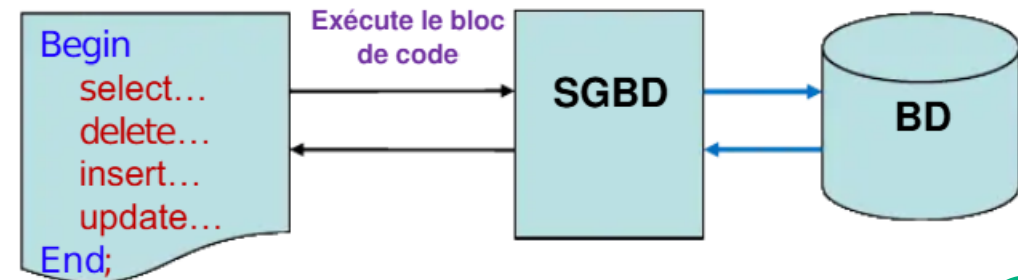
► Lorsque le serveur reçoit un appel pour exécuter un programme PL/SQL, la version compilée du programme est chargée en mémoire puis exécutée par les moteurs PL/SQL et SQL.



► En **SQL**, les requêtes sont exécutées de manière séquentielle. Chaque instruction SQL est exécutée séparément, une après l'autre.



► Avec **PL/SQL**, vous pouvez écrire des blocs de code qui contiennent plusieurs instructions SQL, Ces blocs peuvent exécuter plusieurs





```
[DECLARE Optionnelle
    -- Déclaration de variables, constantes, exceptions, curseurs ]
BEGIN
    -- Section obligatoire contenant des commandes exécutables
    -- Instructions SQL et PL/SQL
    -- imbrication de blocs) Obligatoire
[EXCEPTION Optionnelle
    -- Traitement des exceptions (gestion des erreurs) ]
END;
```

/ ← A ajouter obligatoirement dans l'exécution d'un script

Remarque : Le principe d'imbrication de bloc PL/SQL autorise l'emboîtement des blocs après BEGIN. Il permet une meilleure lisibilité des programmes décomposés en sous-bloc



Exemple:

```
-- Activation de la sortie serveur
SET SERVEROUTPUT ON
-- Début du bloc PL/SQL
BEGIN
  -- Affichage d'un message
  DBMS_OUTPUT.PUT_LINE('Bonjour, Voici mon premier code PL/SQL!');
END;
/
```

Les déclarations et les instructions doivent être terminées par un point-virgule (;).

- **SERVEROUTPUT ON:** Permettre à SQL d'afficher les résultats générés
- **DBMS_OUTPUT.PUT_LINE():** une procédure utilisée pour afficher un message
- **DBMS_OUTPUT.PUT():** une procédure utilisée pour afficher le contenu d'une variable



- **--:** Ecrire un commentaire sur une seule ligne
- **/* et */:** Créer un commentaire multi-lignes

-- Ceci est un commentaire sur une seule ligne

--les doubles chevrons « et » représentent une étiquette.

« Bloc1 »

DECLARE

A INTEGER ;
B INTEGER ;

BEGIN

....

DECLARE

A INTEGER;
C INTEGER;

BEGIN

.... IF Bloc1.A = A THEN END IF ; ...

EXCEPTION

...

END;

EXCEPTION

WHENTHEN;
WHENTHEN;

END;

/*

Ceci est un commentaire
sur plusieurs lignes

***/**



La partie déclarative dans un bloc PL/SQL, peut comporter trois types de déclarations, elle est délimitée par les mots clés **DECLARE** et **BEGIN**

Les types de déclarations possibles dans cette partie sont :

- Déclaration des **variables** et des **constantes**,
- Déclaration des **curseurs**,
- Déclaration des **exceptions**.

PL/SQL supporte les types du langage SQL (CHAR, VARCHAR2, NUMBER, DATE) et offre des types supplémentaires pour déclarer des variables et des constantes ainsi que des sous types.



Une variable se déclare de la sorte:

nom type [:= initialisation] ;

:= utilisée pour assigner une valeur à une variable (facultatif)

Il est important de noter que les déclarations multiples sont interdites
→ une seule déclaration par ligne

Les identificateurs de variables et de constantes doivent être différents des noms de colonnes

PL/SQL n'est pas sensible à la casse:

- NOM_VARIABLE NUMBER ;
- Nom_Variable Number ;
- nom_variable number ;



Variables PL/SQL:

Types scalaires: recevant une seule valeur

- **INTEGER, REAL, DATE, BOOLEAN + %TYPE**

+types SQL: tous les types SQL sont utilisables en PL/SQL (VARCHAR, NUMBER, etc.)

Types composites: %ROWTYPE, RECORD

- Variables non PL/SQL: définies sous SQL*Plus (de substitution et globales)

Exemple de déclaration de variables :

```
DECLARE
  nom_client VARCHAR2(50); -- Variable de type chaîne de caractères
  age_client  NUMBER := 30; -- Variable de type nombre, initialisée à 30
  date_commande DATE; -- Variable de type date
BEGIN
  ...
END;
```

```
Declare
age integer;
nom varchar2(30);
dateNaissance date;
ok boolean := true;
```

```
declare
date_naissance DATE := 'DD/MM/YYYY';
compteur integer default 0;
id char(5) not null ;
```



Déclaration de variables

Type de données PL/SQL	
Types scalaires	
BOOLEAN	Variable booléenne (TRUE, FALSE, NULL)
BINARY_INTEGER	Entier signé de -2^{31} à $2^{31}-1$ stocké en binaire, directement utilisable sans conversion.
NUMBER(n,m)	Valeur numérique décimale ou entière. La maximum de n est 38. m varie entre -84 et 127.
INT, INTEGER, SMALLINT, DEC, REAL, DECIMAL	Des sous-types de NUMBER
NATURAL	Sous-ensemble de BINARY_INTEGER des entiers de 0 à 2147483647
POSITIVE	Sous-ensemble de BINARY_INTEGER des entiers de 1 à 2147483647
ROWID	Identificateur interne unique de chaque n-uplet dans la base. Il est stocké en format binaire de longueur fixe. Peut être converti en Hexadécimale par la fonction ROWIDTOCHAR qui retourne une chaîne de 18 caractères de la formeBBBBBBBB.RRRR.FFFF représentant le numéro du bloc dans la base (BBBBBBBB), le numéro de ligne (RRRR) dans le bloc et (FFFF) le numéro du fichier. (Ex. <code>Select ROWID, nomEmp from Employe where numDep = 20;</code>)



Type de données PL/SQL	
Types composés	
TABLE	Tableau dynamique dont les éléments peuvent être soit de type scalaire ou composé accessible à travers un indice de type BINARY_INTEGER .
RECORD	Pour déclarer un type enregistrement qui peut être défini de deux façons, soit en définissant ses champs, soit en se référant à une structure de données (table de la BD ou curseur)



Déclaration de variable basée sur une colonne Table

%TYPE tous Attribut utilisé pour déclarer une variable ou un paramètre de procédure/fonction en se basant sur le type de données d'une colonne existante dans une table

Supposons une table **clients** avec une colonne **nom** de type **VARCHAR2** :

```
DECLARE
  nom_client clients.nom%TYPE;
BEGIN
  ...
END;
```

Déclaration de la variable **nom_client** basée sur le type de la colonne **nom** de la table **clients**

Remarque: NOT NULL ne s'applique pas aux variables déclarées avec %TYPE.



Déclaration de variables qui ont le même type qu'un n-uplet (ligne)

%ROWTYPE Clause utilisée pour définir des variables qui ont la même structure qu'une ligne spécifique d'une table de base de données existante

```
DECLARE  
Variable Nomtable%ROWTYPE;
```

Par exemple, supposons que vous ayez une table **employe** avec des colonnes telles que **emp_id**, **emp_name**, **salary**, etc.

```
DECLARE  
  v_emp employe%ROWTYPE;  
BEGIN  
  v_emp.emp_id := 10;  
  v_emp.emp_name := 'John Doe';  
  v_emp.salary := 5000;  
  ...  
END;  
/
```



Déclaration d'un type d'enregistrement

Un enregistrement Structure de données personnalisée qui peut contenir plusieurs champs, chacun avec son propre type de données

- Un enregistrement peut être défini de deux façons, soit en définissant ses éléments, soit en se référant à une structure de données (table de la base ou curseur)

```
DECLARE
-- Déclaration d'un type d'enregistrement personnalisé
TYPE type_client IS RECORD (
    nom    VARCHAR2(50),
    age     NUMBER,
    adresse VARCHAR2(100)
);
-- Déclaration d'une variable de type d'enregistrement
info_client type_client;
BEGIN
...
END;
```



Déclaration de variables & constantes

```
NomVariable [CONSTANT]
{ type | variable%TYPE | Table.Colonne%TYPE |table%ROWTYPE }
[NOT NULL] [ { := | DEFAULT} expression PL/SQL ] ;
```

CONSTANT Mettre une valeur à une variable lors de sa définition

NOT NULL Pose une contrainte en ligne sur la variable, elle faut être initialiser lors de sa déclaration

DEFAULT Permet d'initialiser une variable (équivalent à l'affectation :=)

La récupération de la valeur d'une colonne se fait par l'opérateur **into**.

✓ Exemple:

```
select nomEtudiant into vnom from ETUDIANT where matEtudiant = '2016-0005';)
```



Variables

```
DECLARE
  v_dateNaissance DATE;
/* équivaut à
  v_dateNaissance DATE := NULL;
*/
  v_capacité     NUMBER(3) := 999;
  v_téléphone    CHAR(14)
                NOT NULL := '06-76-85-14-89';
  v_trouvé       BOOLEAN NOT NULL := TRUE;
BEGIN
...
```

Constantes et expressions

```
DECLARE
  c_pi CONSTANT NUMBER := 3.14159;
  v_rayon  NUMBER := 1.5;
  v_aire   NUMBER := c_pi * v_rayon**2;
--
  v_groupeSanguin CHAR(3) := 'O+';
/* équivaut à
  v_groupeSanguin CHAR(3) DEFAULT 'O+';
*/
  v_dateValeur DATE := SYSDATE + 2;
BEGIN
...
```



Il n'est pas possible d'affecter une valeur nulle à une variable définie `NOT NULL` (l'erreur renvoyée est l'exception prédéfinie `VALUE_ERROR`).
La contrainte `NOT NULL` doit être suivie d'une clause d'initialisation.



- ▶ Les tableaux en PL/SQL sont conçus comme des tables de la BD.
 - Ils possèdent une clé primaire (index) pour accéder aux lignes du tableau.
 - Un tableau ne possède pas de limites de taille.

De cette façon, le nombre d'éléments d'un tableau va croître dynamiquement.

- ▶ La déclaration d'un tableau se fait en deux étapes :
 - Déclaration d'un **nouveau type tableau**
 - Déclaration de la **variable sur le type défini**



► Déclaration d'un nouveau type **tableau**

```
TYPE Nom_Type IS TABLE OF  
{ typecolonne | variable%TYPE | table.colonne%TYPE } [NOT  
NULL]  
INDEX BY { BINARY_INTEGER | PLS_INTEGER };
```

► Déclaration d'une variable de type **Nom_Type**

```
Nom_Variable Nom_Type ;
```



Action	Syntaxe	Observation
Accès aux éléments du tableau	Nom_Tab(Valeur_clé_primaire)	Valeur_Clé_primaire de type binary_integer
Affectation d'une constante ou d'une expression PL/SQL	Nom_Tab(Valuer_clé_primaire) := Constante Expression PL/SQL	
Pour se référer à la valeur d'une propriété	Nom_Tab.Nom_Propriété	



Les propriétés d'un tableau PL/SQL	
Exists(n)	True si le nième élément existe dans le tableau
Count	Fournit le nombre d'éléments du tableau PL/SQL
First Last	Retournent respectivement le plus petit et le plus grand indice du tableau (NULL si le tableau est vide)
Prior(n) Next(n)	Retournent respectivement l'élément précédent/suivant le nième élément du tableau
Delete(n) Delete(n,m)	Supprime un ou plusieurs éléments du tableau



► Exercice:

Soit le schéma suivant:

FILM(ID_FILM, TITRE, #IDMESC, ANNEE, COUT, RECETTE)

ARTISTE(ID, NOM, PRENOM, DATENAISS)

- Créer un bloc anonyme qui permet:
- Pour un id de FILM donné:
 - Récupère le titre du film correspondant,
 - Récupère les informations du metteur en scène (IDMESC) dans la table ARTISTE
 - Affiche le titre et le nom à l'aide de DBMS_OUTPUT.PUT_LINE



► L'implémentation du nouveau **type composé** nommé **RECORD** a permis de **déclarer des variables de type enregistrement**

```
TYPE Nom_Type IS RECORD (  
  champ { typechamp | table.column%TYPE} [NOT NULL],  
  champ { typechamp | table.column%TYPE} [NOT NULL], ....)
```

```
Nom_Variable  Nom_Type ;
```

DECLARE

-- Déclaration d'un type d'enregistrement personnalisé

TYPE type_client **IS RECORD** {

nom VARCHAR2(50),

age NUMBER,

adresse VARCHAR2(100)

};

-- Déclaration d'une variable de type d'enregistrement

info_client type_client;

BEGIN

...

END;

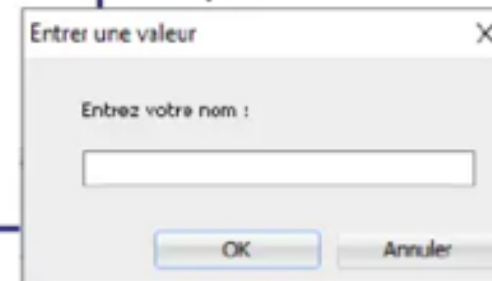


► La procédure **ACCEPT** est utilisée pour lire des valeurs saisies par l'utilisateur pendant l'exécution du programme PL/SQL

ACCEPT variable_name **PROMPT** 'texte'

Exemple 1 :

```
ACCEPT nom PROMPT 'Entrez votre nom : ' ;  
DECLARE  
    v_nom VARCHAR2(50);  
BEGIN  
    v_nom := '&nom';  
    DBMS_OUTPUT.PUT_LINE('Bonjour, ' || v_nom);  
END;  
/
```





- L'affectation d'une expression se fait par l'opérateur :=
- Outre les conversions explicites par les fonctions prédéfinies (TO_CHAR, TO_NUMBER, TO_DATE, ...), PL/SQL peut effectuer des conversions implicites lors des affectations ou évaluations d'expressions.

- ♣ Affectation d'un nombre dans une chaîne.

Ex. message VARCHAR2(20) := 1200;

- ♣ Affectation d'une chaîne contenant un nombre.

Ex. salaire number (7,3):= '1200.500';

- ♣ Conversion implicite entre DATE et CHAR.

Ex. aujourd'hui VARCHAR2(9):= SYSDATE;
Vdate DATE := '16-Sep-2021';

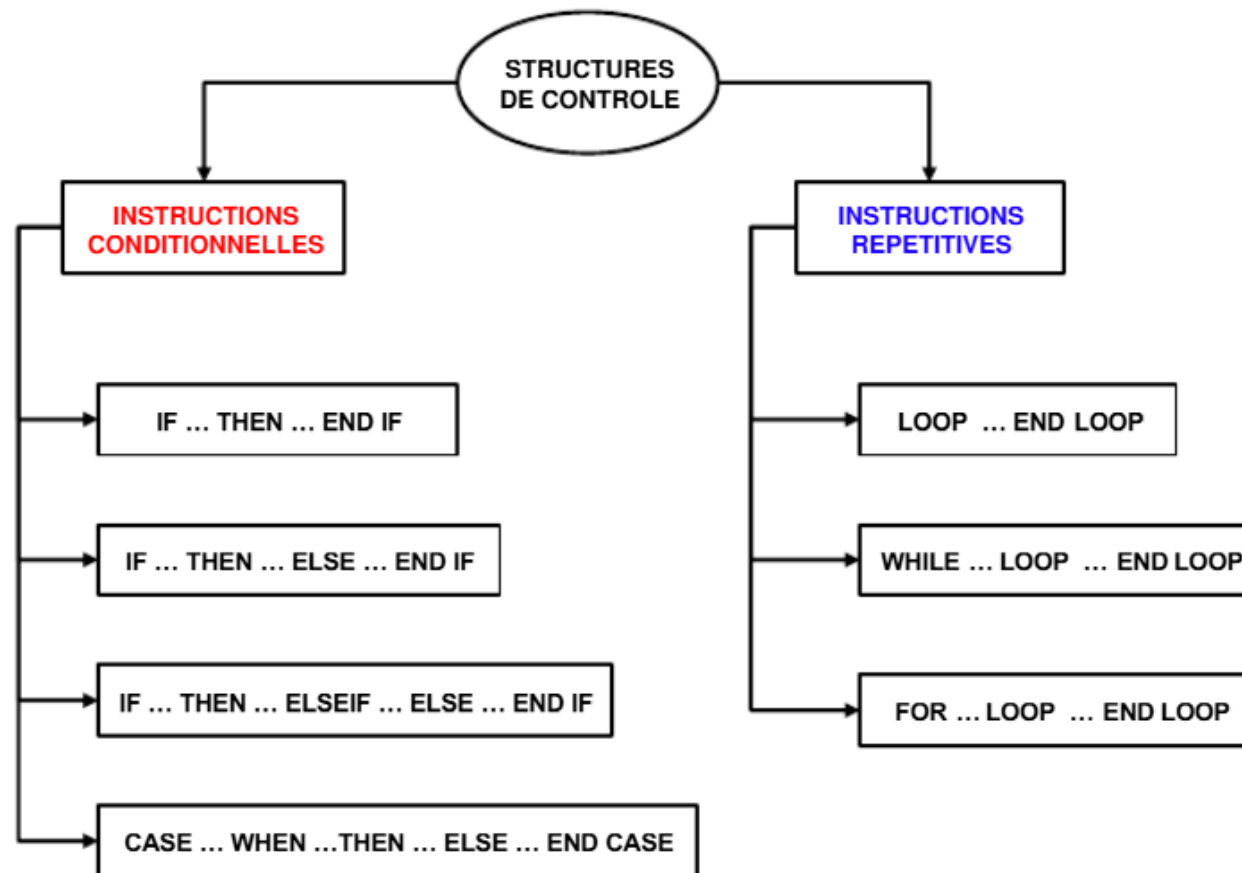


► L'évaluation des expressions repose sur les règles de priorité des opérateurs présentés de la plus haute vers la plus faible priorité :

Opérateurs	Opération
** , NOT	Puissance, négation logique
+ , -	Identité, inversion de signe
* , /	Multiplication, division
+ , - , 	Addition, soustraction, concaténation
< , <= , > , >= , = , !=	Opérateurs de comparaison
IS NULL , LIKE , BETWEEN , IN	Test de nullité, comparaison de chaîne, intervalle de valeur, appartenance
AND	ET logique
OR	OU logique



► Les principales structures de contrôle de PL/SQL sont:





- Utilisation de **ELSE** pour les structures imbriquées

```
IF condition1_PLSql
  THEN Traitement1
  [ELSE]
    [IF condition2_PLSql
      THEN Traitement2
      [ELSE TraitementN]
    END IF; ]
END IF;
```

- Utilisation de **ELSIF** pour les structures imbriquées

```
IF condition_PLSql
  THEN commandes
  [ELSE commandes]
  [ELSIF condition_PLSql
    THEN commandes
    [ELSE commandes]]
END IF;
```

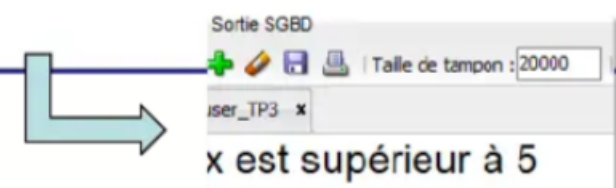



Les traitements conditionnels : L'alternative

► Exemple

```
DECLARE
    m INTEGER := 12;
BEGIN
    IF (m=1) THEN
        dbms_output.put_line('Janvier');
    ELSIF (m=2) THEN
        dbms_output.put_line('Fevrier');
        ...
    ELSIF (m=12) THEN
        dbms_output.put_line('Décembre');
    ELSE
        dbms_output.put_line('Numéro de mois erroné');
    END IF;
END;
```

```
DECLARE
    x NUMBER := 10;
BEGIN
    IF x > 5 THEN
        DBMS_OUTPUT.PUT_LINE('x est supérieur à 5');
    ELSE
        DBMS_OUTPUT.PUT_LINE('x est inférieur ou égal à 5');
    END IF;
END;
```

A screenshot of the SQL Developer output window. The window title is "Sortie SGBD". It shows the execution of the PL/SQL code, resulting in the output "x est supérieur à 5". The window also displays the username "user_TP3" and the buffer size "Taille de tampon : 20000".



```
[<<label>>] CASE sélecteur
    WHEN expression1 THEN Trait1 ;
    WHEN expression2 THEN Trait2 ;
    ...
    ELSE TraitN;
END CASE [label];
```

► Exemple

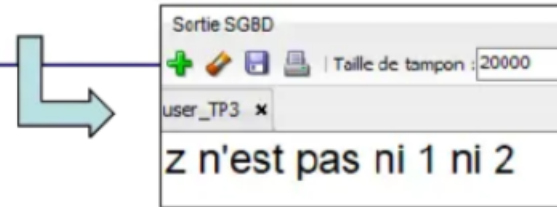
```
DECLARE
    m INTEGER := 12;
BEGIN
    <<mois>> CASE m
        WHEN 1 THEN dbms_output.put_line('Janvier');
        WHEN 2 THEN dbms_output.put_line('février');
        ...
        WHEN 12 THEN dbms_output.put_line('décembre');
        ELSE dbms_output.put_line('mois erroné');
    END CASE mois;
END;
```



```
[<<label>>] CASE sélecteur  
    WHEN expression1 THEN Trait1 ;  
    WHEN expression2 THEN Trait2 ;  
    ...  
    ELSE TraitN;  
END CASE [label];
```

► Exemple

```
DECLARE  
    z NUMBER := 3;  
BEGIN  
    CASE z  
        WHEN 1 THEN  
            DBMS_OUTPUT.PUT_LINE('z est égal à 1');  
        WHEN 2 THEN  
            DBMS_OUTPUT.PUT_LINE('z est égal à 2');  
        ELSE  
            DBMS_OUTPUT.PUT_LINE('z n'est pas ni 1 ni 2');  
    END CASE;  
END;
```





La structure Loop End Loop

```
BEGIN  
[ <<Label>> ] LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

La structure While

```
[ <<Label>> ]  
WHILE condition_PLSql LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

```
DECLARE  
compteur NUMBER := 1;  
BEGIN  
  WHILE compteur <= 5 LOOP  
    DBMS_OUTPUT.PUT_LINE('compteur : ' || compteur);  
    compteur := compteur + 1;  
  END LOOP;  
END;
```

A screenshot of a SQL script execution window titled "Sortie de script". It shows the output of the PL/SQL code, which is a list of numbers from 1 to 5, each preceded by "compteur :".

```
Sortie de script  
Sortie SQLDB  
user_TP3  
compteur : 1  
compteur : 2  
compteur : 3  
compteur : 4  
compteur : 5
```



```
[ <<Label>> ]  
FOR compteur IN [REVERSE] val_debut .. val_fin LOOP  
...  
Instructions  
...  
END LOOP [Label];  
END;
```

```
DECLARE  
    total NUMBER := 0;  
BEGIN  
    FOR i IN 1..5 LOOP  
        total := total + i;  
    END LOOP;  
    DBMS_OUTPUT.PUT_LINE('Total : ' || total);  
END;
```



```
--Etiquette  
<< Label >>  
.....  
--instruction GOTO  
GOTO Label ;
```

► Exemple

```
DECLARE  
    i number :=1;  
    f number:=1;  
    n number:=5;  
  
BEGIN  
    <<instruction I>>  
    f := f * i;  
    i:=i+1;  
    if (i <= n) then  
        GOTO instruction I;  
    End if;  
    dbms_output.put_line('Factoriel de '||n||' égale : ' || f);  
END;
```



Pour quitter plusieurs boucles imbriquées, il faut tout d'abord **étiqueter les boucles** puis **utiliser l'option étiquette de la commande EXIT**.

► Exemple

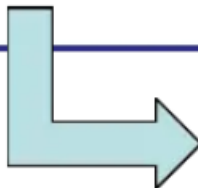
```
<<boucleI>>  
LOOP  
...  
    LOOP  
    ...  
    EXIT WHEN ...  
    EXIT boucleI WHEN ...  
    ...  
    END LOOP;  
...  
END LOOP boucleI;
```



Exercice

Ecrire un programme PL/SQL qui affiche les nombres de 1 à 10 en utilisant une boucle For. Si le nombre est pair, affichez « Pair », sinon affichez « Impair »

```
DECLARE
  v_num NUMBER;
BEGIN
  FOR v_num IN 1..10 LOOP
    IF MOD(v_num, 2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE(v_num || ' : Pair');
    ELSE
      DBMS_OUTPUT.PUT_LINE(v_num || ' : Impair');
    END IF;
  END LOOP;
END;
/
```



Sortie SQL*Plus

user_TP3
1 : Impair
2 : Pair
3 : Impair
4 : Pair
5 : Impair
6 : Pair